"Echo" I2S Wake-Word Offloader for Croc SoC

VLSI2 Final Project Report

Prepared by:

Oleg Tsyplakov (otsyplakov@student.ethz.ch)
Anna Lubbers (alubbers@student.ethz.ch)

Submitted to:

Integrated Systems Laboratory

Department of Information Technology and Electrical Engineering
ETH Zürich

June 29, 2026

Contents

1 Introduction & Project Overview **2**
- 1.1 Motivation . . . 2
- 1.2 Goal . . . 2

2 System Architecture & RTL Design **2**
- 2.1 Voice Activity Detection Core . . . 2
- 2.2 I2S Wrapper . . . 3
- 2.3 VAD Subsystem . . . 4
- 2.4 CSR Wrapper . . . 4

3 Functional Verification & Testing **4**
- 3.1 Golden Model . . . 4
- 3.2 Isolated Module Verification . . . 6
- 3.3 Integrated Module Verification . . . 6
- 3.4 Post-Layout Verification . . . 6

4 Backend Implementation **6**
- 4.1 Floorplanning and Placement . . . 6
- 4.2 Clock Tree Synthesis . . . 7
- 4.3 Routing and Finishing . . . 7
- 4.4 DRC . . . 7
- 4.5 LVS . . . 7

5 Evaluation Setup & Performance Results **7**
- 5.1 Energy Savings Estimate . . . 7
- 5.2 Improvement . . . 8
- 5.3 Interpretation . . . 8

6 Conclusion **9**

1 Introduction & Project Overview

This report details the design and implementation of a hardware-based Voice Activity Detection (VAD) subsystem developed as a wake-word offloader for the Croc SoC. The subsystem is integrated as a user-domain peripheral, communicating with the SoC's CVE2 core over the OBI bus. The design was carried through RTL design, physical implementation (synthesis, floorplanning, place & route, clock-tree synthesis, and finishing), and final verification (DRC and LVS).

1.1 Motivation

Voice-activated devices require continuous acoustic monitoring to detect wake words or voice commands. However, keeping the main processor core continuously active solely to ingest and analyze quiet or empty audio streams introduces a substantial and largely unnecessary power penalty, since speech activity occupies only a small fraction of a typical listening window. To minimize power consumption, a dedicated, hardware-efficient peripheral is required to autonomously stream audio, calculate acoustic features, and isolate periods of human speech without requiring the main core to be active. This offloader keeps the main SoC domain in a low-power sleep state, asserting a hardware interrupt to wake the core only when valid speech is detected. Decoupling continuous audio monitoring from the main core in this way is the central objective of the project: the core's energy consumption should scale with how often speech actually occurs, not with how long the system listens.

1.2 Goal

The goal of this project is to implement a complete hardware audio-ingestion and feature-extraction pipeline that can operate independently of the main core. The proposed architecture acts as a dedicated user-domain peripheral that autonomously captures a 16-bit serial I^2S audio stream, safely transfers it across the mesochronous boundary between the I^2S sampling clock and the system clock, stores it in a double-buffered (ping-pong) frame buffer, and uses a dedicated hardware digital signal processing (DSP) engine to evaluate Short-Time Energy and Zero-Crossing Rate (ZCR) over each captured frame. Both detection thresholds are runtime-configurable through a memory-mapped control and status register (CSR) interface, allowing the wake sensitivity to be tuned without re-synthesizing the design. Upon crossing the programmed thresholds, the system asserts a single-cycle wake-up interrupt to the Croc SoC, at which point the main core can transition out of its sleep state to perform further processing.

2 System Architecture & RTL Design

The "Echo" system consists of multiple hierarchical layers, mirroring the way it is integrated into Croc's user domain. The outermost layer is the `vad_csr_wrapper`, which is the module directly instantiated in the user domain and exposed to the OBI crossbar. It contains three control and status registers (CSRs) alongside an instance of the VAD subsystem. The VAD subsystem itself integrates two functional blocks: the I^2S wrapper, responsible for ingesting and buffering the serial audio stream, and the VAD core, responsible for the digital signal processing (DSP) path that ultimately decides whether speech is present. Figure 1 depicts the block diagram of the full system and its connection to the CVE2 core via the OBI crossbar.

2.1 Voice Activity Detection Core

The VAD core is the heart of "Echo". For each captured audio frame, it computes the samples' short-time energy and zero-crossing rate (ZCR) and compares both metrics against independently configurable thresholds. The energy threshold must be exceeded for a frame to be con-

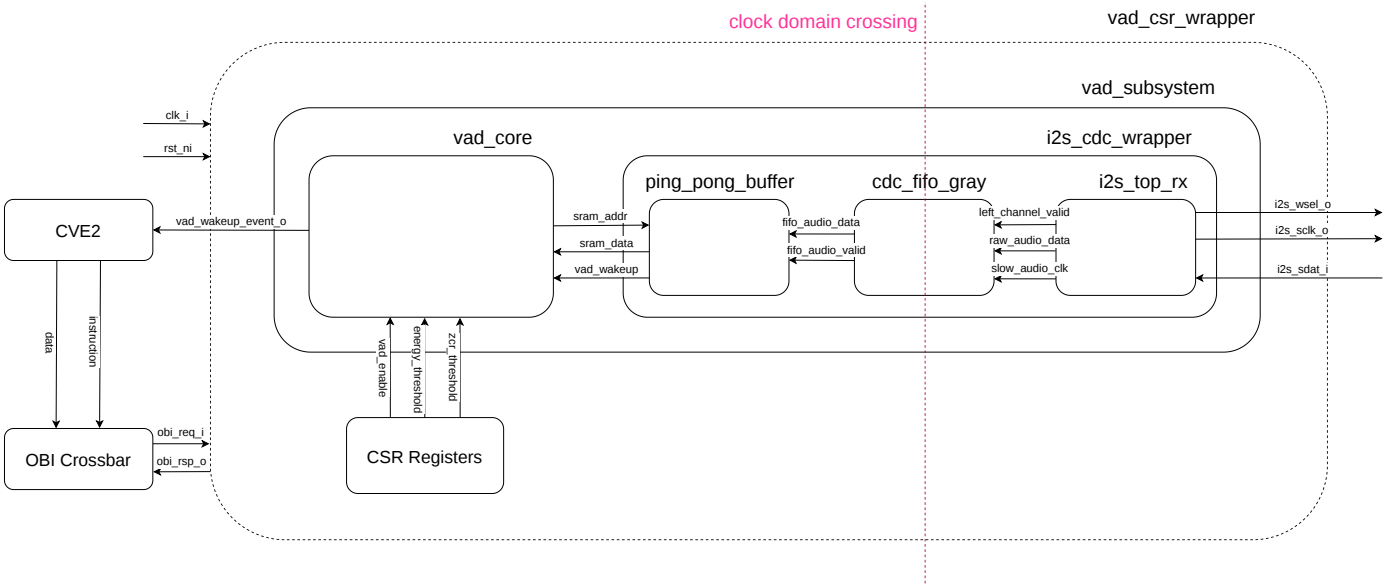


Figure 1: Block diagram of the implemented modules and how they are connected to the CVE2 and OBI crossbar.

sidered loud enough to plausibly contain deliberate speech, while the zero-crossing rate must remain below its threshold, since high-frequency noise and unvoiced/fricative sounds tend to produce a disproportionately high zero-crossing count relative to voiced speech. A frame is only classified as speech when both conditions are satisfied simultaneously, which provides a simple but effective two-feature discriminator between voice activity and background noise without requiring a full spectral analysis.

2.2 I2S Wrapper

As many low-power microphones use the I^{2}S protocol to transmit audio data, this was the interface chosen for "Echo". The microphone-side serial clock is derived locally from the system clock through an internal integer clock divider (`CLK_DIV_FACTOR`).This makes the crossing between the audio-sampling domain and the system clock domain *mesochronous* rather than fully asynchronous; nevertheless, a Gray-coded asynchronous FIFO is used for the crossing to guarantee correctness even if the divider ratio or clocking topology is changed in a future revision, rather than relying on the fixed-phase relationship explicitly. The pipeline from microphone to VAD core proceeds as follows:

- **I^{2}S Top Receiver:** This submodule deserializes the 16-bit I^{2}S data stream from the microphone, generating the serial clock (`sclk`) and word-select (`wsel`) signals that drive it, and outputs each decoded sample together with a channel flag identifying whether it belongs to the left or right audio channel. Since the VAD pipeline only operates on a single (mono) channel, only samples flagged as belonging to the left channel are forwarded downstream; right-channel samples are discarded at this stage. Since this is a standard component, we took an existing implementation from MatejGomboc's Github and integrated it into our system.
- **FIFO CDC Buffer:** A Gray-coded asynchronous FIFO, taken from the existing repository of verified common cells, safely transfers each left-channel audio sample from the slow, microphone-derived clock domain into the system clock domain.

- **Ping-Pong Buffer:** Once in the system clock domain, samples are written sequentially into one of two SRAM-backed frame buffers (banks). While one bank is being filled with incoming audio, the VAD core is free to read the other, already-complete bank through an addressed read interface. When a bank is filled (after `FRAME_SIZE` samples), the buffer swaps the active write and read banks and asserts a wake-up pulse to the VAD core, indicating a fresh frame is ready for analysis. This double-buffering scheme decouples audio capture from VAD processing, ensuring that incoming audio is never dropped or stalled regardless of how long the VAD core takes to evaluate the previous frame.

2.3 VAD Subsystem

The VAD core and the I^2S pipeline are combined into the VAD subsystem, which instantiates both blocks and wires them together internally. The ping-pong buffer's wake-up pulse (`vad_wakeup`, internal to the subsystem) signals the VAD core that a new frame is ready to be read and evaluated; this is distinct from `vad_wakeup_event_o`, the subsystem's top-level output, which is only asserted once the VAD core has actually classified a frame as containing speech. In other words, `vad_wakeup` triggers *evaluation* of a frame, while `vad_wakeup_event_o` reports the *result* of that evaluation to the rest of the SoC.

2.4 CSR Wrapper

The outermost module, `vad_csr_wrapper`, contains three memory-mapped control and status registers: an enable bit, an energy threshold, and a zero-crossing-rate threshold. This wrapper is the block directly attached to the OBI bus; it decodes incoming OBI requests, handles register reads and writes, and forwards the resulting register values into the VAD subsystem as plain control signals. Because the CVE2 core accesses these registers as ordinary memory-mapped addresses over the OBI data bus, software running on the core can reconfigure "Echo's" sensitivity at runtime through standard load and store instructions. This design allows the wake-word detector to be tuned to a specific speaker's voice level or background noise environment, and even retuned during operation, without requiring the design to be re-synthesized.

3 Functional Verification & Testing

To verify our "Echo" the design was verified after each relevant stage. A Python golden model was used to generate stimuli and expected outputs.

3.1 Golden Model

To validate the VAD core's classification logic independently of the RTL implementation, a Python golden model was developed that reproduces the exact feature-extraction and decision logic implemented in hardware: short-time energy (STE) and zero-crossing rate (ZCR), evaluated frame-by-frame and compared against the same dual-threshold criterion described in Section 2.

Three audio stimuli were recorded for evaluation: a single-word sample ("hallo"), a multi-word sample ("test, testing, hello"), and a silence-only sample used as a negative control to verify the model does not produce false positive detections in the absence of speech.

For each recording, the script first splits the waveform into non-overlapping frames of `frame_size` samples (320 samples per frame, matching the hardware's `FRAME_SIZE` parameter). For each frame, the short-time energy is computed as the sum of squared sample values,

$$E_n = \sum_{k=0}^{N-1} x_n[k]^2, \tag{1}$$

where $x_n[k]$ is the k-th sample of frame n and N is the frame size, computed in 64-bit precision to avoid overflow during accumulation. The zero-crossing rate is computed by counting the number of sign changes between consecutive samples within the frame,

$$Z_n = \sum_{k=1}^{N-1} |\text{sign}(x_n[k]) - \text{sign}(x_n[k-1])| \,. \tag{2}$$

Rather than using a fixed energy threshold, the model derives one adaptively from the recording itself: the average energy of the first `noise_frames` frames (the first 25 frames, assumed to contain only background noise/silence at the start of the recording) is taken as a noise floor estimate, and the energy threshold is set to a fixed multiple of this noise floor. The zero-crossing rate threshold is currently fixed at 20 counts per frame. Each frame is classified as speech only if its energy exceeds the adaptive energy threshold *and* its zero-crossing rate remains at or below the ZCR threshold, mirroring the AND-combination of both criteria implemented in the VAD core. The raw audio data, short-time energy and zero-crossing rate of the audio sample containing the word "hallo" can be seen in Figure 2.

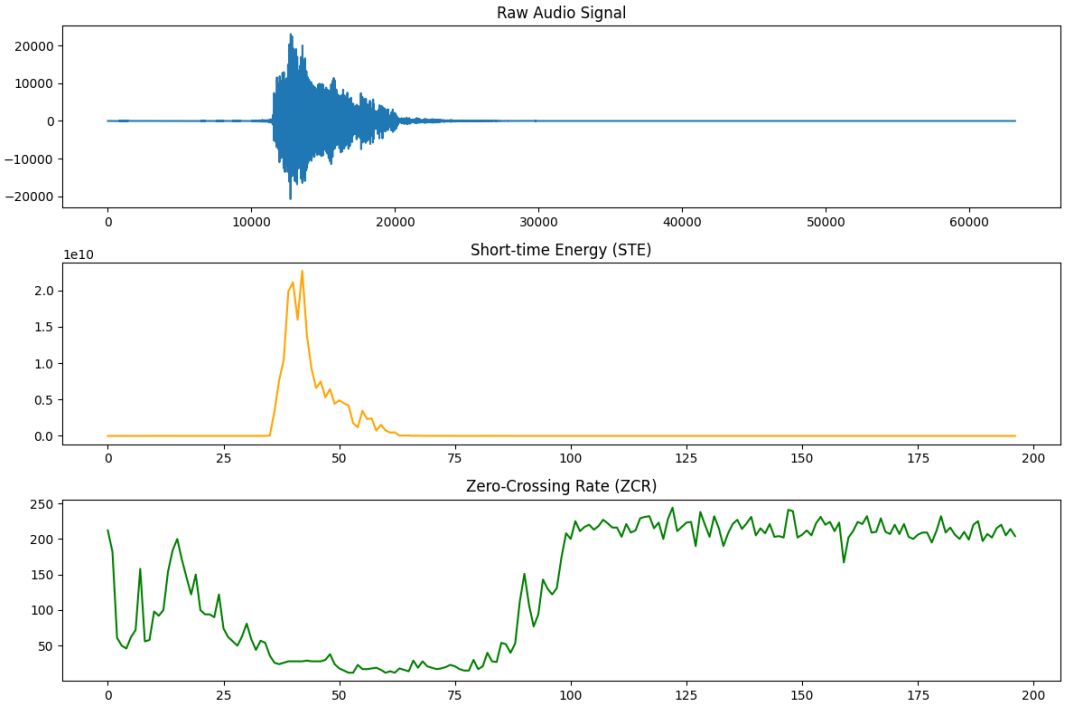


Figure 2: Raw audio data, short-time energy and zero-crossing rate of the audio sample containing the word "hallo".

The script produces two output files used directly as stimulus and reference for RTL simulation:

- **Audio stimulus (`audio_stimulus_*.hex`):** the raw 16-bit PCM samples of the recording, one per line in hexadecimal, used to drive the testbench's simulated I^2S microphone model exactly as a real microphone would deliver them.
- **Golden frame indices (`golden_frames.hex`):** the indices of all frames the Python model classified as speech, used to verify that the RTL's actual `vad_wakeup_event_o` is asserted correctly during simulation.

This approach decouples verification of the *detection algorithm* (energy/ZCR thresholding and its parameter tuning) from verification of the *hardware implementation* (correct frame

buffering, clock-domain crossing, and DSP datapath timing): any mismatch between the golden frame indices and the RTL's actual wake events during simulation can be attributed to an implementation bug rather than a flaw in the detection criterion itself, since both sides apply the identical mathematical rule to the identical input samples.

3.2 Isolated Module Verification

The RTL was first verified in isolation using `tb_vad_subsystem.sv`, instantiating only the `vad_subsystem` module with hardcoded thresholds, before the CSR wrapper existed. A behavioral I^2S microphone model streams the audio stimulus exported by the Python golden model (Section 3.1) into the DUT, correctly replicating I^2S timing and left-channel-only sampling. A self-checking scoreboard compares each `speech_detected_o` assertion against the golden frame indices, classifying results as an exact match, an early trigger, a missed detection, or a trailing "hangover" beyond the last expected frame.

3.3 Integrated Module Verification

Once integrated into Croc's user domain, the design was re-verified at the SoC level using `tb_croc_soc.sv`, driven through Croc's JTAG-based verification IP. This was done in two stages: first with hardcoded CSR values, to confirm correct operation once embedded in the full SoC and connected to the OBI crossbar; then with the `vad_csr_wrapper` enabled, where firmware configures the energy and ZCR thresholds at runtime via JTAG register writes, exercising the full software-to-hardware configuration path. The same audio-stimulus and golden-frame scoreboarding methodology from the isolated testbench was reused, with the VAD wake-up event monitored directly inside the user domain.

Interrupt Handling: The wake-up interrupt was confirmed to reach the CVE2 core, but full firmware-side handling of the interrupt (e.g., waking the core from sleep) was not completed, as this falls under software rather than hardware verification and turned out to be rather complicated. When trying to implement this, there were errors thrown, such as `CVE2DontSkipExceptionReq` or `CVE2SetExceptionPCOnSpecialReqIfExpected`. Therefore, for now this interrupt pipeline is not fully verified and handled correctly.

3.4 Post-Layout Verification

To ensure functional correctness was preserved throughout the implementation flow, the design was re-verified against the testbench described above at two further stages. First, the gate-level netlist produced by Yosys after synthesis was simulated and confirmed to match the expected behavior. Second, the netlist produced after the Openroad backend flow was verified in the same way, again confirming correct functionality.

4 Backend Implementation

After the integrated module verification detailed in Section 3.3, the backend flow started. First, the circuit was synthesized with Yosys and then floorplanning, routing and finishing was done in Openroad. Lastly, the finished design was tested for design rule violations (DRC) and the layout was compared to the schematic (LVS).

4.1 Floorplanning and Placement

In the floorplanning stage the size of the chip was changed from the $2\,\text{mm} \times 2\,\text{mm}$ of the reference flow to the $2.5\,\text{mm} \times 2\,\text{mm}$ given for our design. In addition the pin placement was adapted to the given template for this project. The two SRAM blocks were kept in the same position as

in the reference flow. The placement of any other components were made automatically with Openroad and the placement script.

4.2 Clock Tree Synthesis

For the clock tree synthesis the cts-script was used. After running the script the clock tree was synthesized and the timing constraints were met, with the worst maximum slack being 0.02 ns.

4.3 Routing and Finishing

Following the clock tree synthesis the wires between the standard cell were routed. After some iterations Openroad could route the chip and nearly achieving no negative slack with having -0.02 ns. This slack then was determined to be negligible and proceeded to generating the output files with the finishing script.

4.4 DRC

Apart from the expected offgrid errors, we had one more DRC error reported: a pitch error in the Metal1 layer. An inverter and a large flipflop were apparently to close together violating the 0.18 µm pitch constraint by 0.01 µm. At first, this seemed like an easy fix. However, it ended up being part of a standard cell and anytime we tried to fix our one DRC error with moving the metal box surrounding a via, we ended up with about 50'000-100'000 more DRC errors. Since the change to our one problematic instance affected all of the other instances of this specific standard cell. To resolve our DRC error we had to isolate this via from the standard cell grouping and then were able to decrease the size of the metal box. After implementing this fix the DRC check run and only showed the 512 DRC errors coming from the pins to the outside of the Croc.

4.5 LVS

Luckily for us, LVS was immediately passed on the first run with no errors, leaving us with a working implementation.

5 Evaluation Setup & Performance Results

5.1 Energy Savings Estimate

Table 1: Post-layout power breakdown for the VAD subsystem and the CVE2 core (`tt` corner).

Power Component	VAD Subsystem (W)	CVE2 Core (W)
Internal	1.589×10^{-2}	2.582×10^{-3}
Switching	1.050×10^{-4}	8.117×10^{-5}
Leakage	2.185×10^{-6}	1.275×10^{-6}
Total	$\mathbf{1.600 \times 10^{-2}}$	$\mathbf{2.664 \times 10^{-3}}$

Post-layout power figures were obtained via Openroad's static power analysis (`report_power`) for both the VAD subsystem and the CVE2 core (`core_wrap`), under the `tt` corner. Table 1 summarizes the internal, switching, and leakage power components for each block. The VAD subsystem, which must run continuously to monitor for wake words, totals 1.600×10^{-2} W, compared to 2.664×10^{-3} W for the CVE2 core in active mode; while clock-gated, the core's power reduces to its leakage component alone, 1.275×10^{-6} W.

To calculate the actualy power dissipated in operation, we estimate the systems duty cycle, meaning the amount of time the CVE2 is turned on. As an example we derive its duty cycle from the golden model's classification of the "hallo" test recording (24 of 197 frames classified as speech), which results in a duty cycle of 12.2%. In a real-life application, there would most likely be longer periods of silence and therefore a shorter duty cycle. The total power consumed under the proposed offload scheme is estimated as

$$P_{\text{offload}} = P_{\text{VAD}} + d \cdot P_{\text{CVE2,active}} + (1 - d) \cdot P_{\text{CVE2,sleep}}, \tag{3}$$

where d is the duty cycle. Substituting the values from Table 1 gives

$$P_{\text{offload}} = 1.600 \times 10^{-2} + 0.122 \times 2.664 \times 10^{-3} + 0.878 \times 1.275 \times 10^{-6} \approx 1.633 \times 10^{-2}\,\text{W}. \tag{4}$$

This compares to a baseline in which the core runs VAD inference continuously,

$$P_{\text{baseline}} = P_{\text{CVE2,active}} = 2.664 \times 10^{-3}\,\text{W}. \tag{5}$$

Contrary to the project's initial goal, this analysis indicates that, at the current implementation's power figures, the always-on VAD hardware's own power consumption outweighs any savings gained from putting the core to sleep, resulting in a net increase in total system power ($P_{\text{offload}}/P_{\text{baseline}} \approx 6.1$) rather than the intended reduction.

From further power analysis, we found out that the majority of power is consumed by the SRAMs in the pingpong buffer. They consume approximately $1.562 \times 10^{-2}\,\text{W}$. So, for a future optimized implementation, this part of the circuit would be the first one to be changed.

5.2 Improvement

After the analysis of the power and timing reports of our implemented design, we decided to improve the RTL implementation and tackle the timing violation. In the old implementation of the ping-pong buffer, the synthesizer simply instantiated a flip-flop for every single bit. This amounted to around 10'000 flip-flops which were all run continuously at the 100 MHz clock. To reduce power consumption while a bank was not being written to, we introduced integrated clock gating. This enables to physically disconnect the 100 MHz wire from the flip-flops that are not actively saving new data. This allows the 100 MHz clock to be physically disconnected from flip-flops that are not actively storing new data, so that a flip-flop only receives a clock edge when it is actually being written to. Since writes occur only a small fraction of the time, this should leave the clock disconnected for roughly 99 % of the time. To confirm that our new implementation remained functionally correct, we re-ran all verification stages and obtained the same results as before. We then repeated the power analysis: The power consumption reported by Openroad for the vad-subsystem is 8.885 mW. This figure is based on OpenROAD's estimate of how often the clock gate is open, so the power consumption in a real application should be even lower. To achieve even lower power consumption, it could be considered to implement dedicated SRAM banks or to not implement a ping-pong buffer at all and therefore risk losing some data.

To address the timing violation and meet the slack requirement, we made two changes to the scripts. First, global-padding was added left and right to the placement script to avoid the DRC errors mentioned earlier. Second, an additional margin of 0.05 ns was added to the final `repair_timing` command in the same script to meet the timing constraint.

5.3 Interpretation

This result highlights that the energy benefit of a hardware wake-word offloader is bounded by the offload hardware's own power cost relative to the core it replaces: offloading is only advantageous when the always-on monitoring circuit is significantly lower-power than the core it allows to sleep.

6 Conclusion

In conclusion, we achieved a working implementation of our voice activity detection accelerator, "Echo", which passed post-layout simulation, DRC, and LVS. The verification process confirmed correct functional behavior, though the interrupt handling at the CVE2 boundary was not implemented yet and should be further developed for the design to be fully robust. Our original implementation had two fundamental limitations: First, timing closure was not fully achieved, because the design exhibited a worst negative slack of $-0.02\,\text{ns}$, as discussed in Section 4.3. Second, as shown in Section 5.1, the system did not achieve any net energy savings, since the VAD subsystem dissipated more power than simply leaving the CVE2 core running continuously. However, in our second iteration the timing issues were solved and the power consumption significantly reduced, although not to an extent where the offloader saves energy as intended. The majority of this overhead still comes from the ping-pong buffer. Ideas to reduce this submodule's energy in the future are stated in Section 5.2.